

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 4 Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

1. Objective

The objectives of this experiment are:

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment, the student will be able to:

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become dominant models for computer vision because they automatically learn hierarchical features from images. The evolution of CNN architectures has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and evolved through AlexNet, VGGNet, GoogleNet and ResNet. Each architecture introduced important innovations that improved image classification performance.

4. Dataset

The dataset used is CIFAR-10.

Property	Value
Training Images	50,000
Testing Images	10,000
Number of Classes	10
Image Size	$32 \times 32 \times 3$

The ten classes are Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

5. Experimental Procedure

5.1 Dataset Preparation

The CIFAR-10 dataset was loaded using TensorFlow/Keras. Pixel values were normalized from $[0, 255]$ to $[0, 1]$, and the class labels were converted to one-hot encoded vectors.

Dataset Dimensions

Dataset	Dimensions
Training Images	(50000, 32, 32, 3)
Training Labels	(50000, 10)
Testing Images	(10000, 32, 32, 3)
Testing Labels	(10000, 10)

Sample CIFAR-10 Images

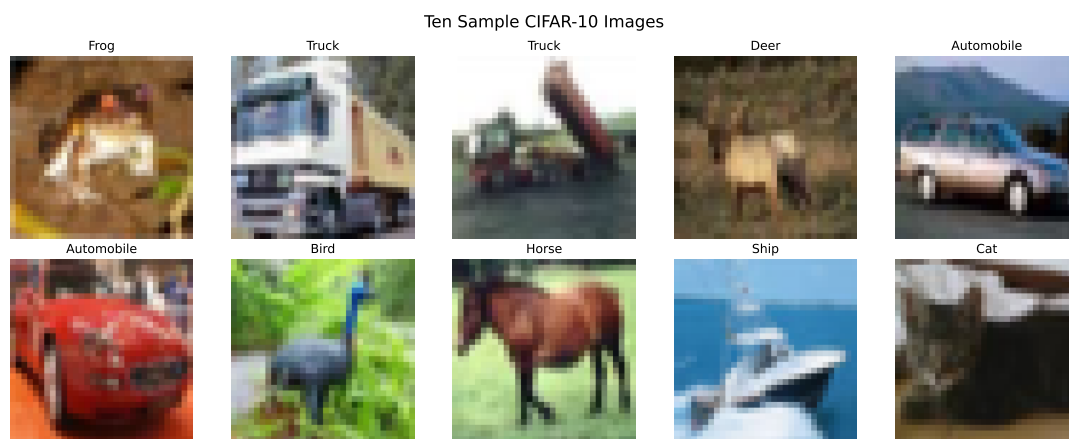


Figure 1: Ten Sample CIFAR-10 Images

Observation:

The example images are representative of the ten CIFAR-10 categories, depicting wide differences in both the objects and backgrounds. The poor resolution of the images makes it hard to differentiate between visually similar categories.

5.2 Transfer Learning using MobileNetV2

The main transfer-learning experiment used MobileNetV2 pretrained on ImageNet. The CIFAR-10 images were resized from 32×32 to 96×96 before being passed to the pretrained network.

Component	Configuration
Input	$32 \times 32 \times 3$
Image Resizing	96×96
Input Rescaling	$[0, 1] \rightarrow [-1, 1]$
Pretrained Base	MobileNetV2, ImageNet weights
Initial Training	Convolutional base frozen
Pooling	Global Average Pooling
Dense Layer	128 units, ReLU
Output Layer	10 units, Softmax

Model Training Configuration

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Initial Epochs	10
Fine-Tuning Epochs	5
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Training Plots

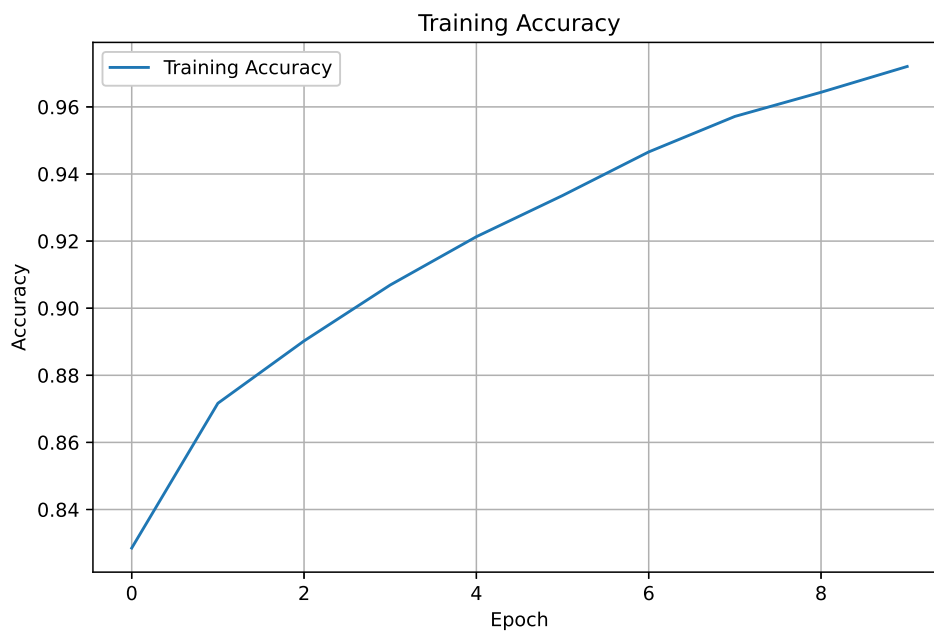


Figure 2: Training Accuracy

Observation:

Accuracy for training grows continuously from about 82.8% up to 97.2%. This suggests that the MobileNetV2 is learning the training data well.

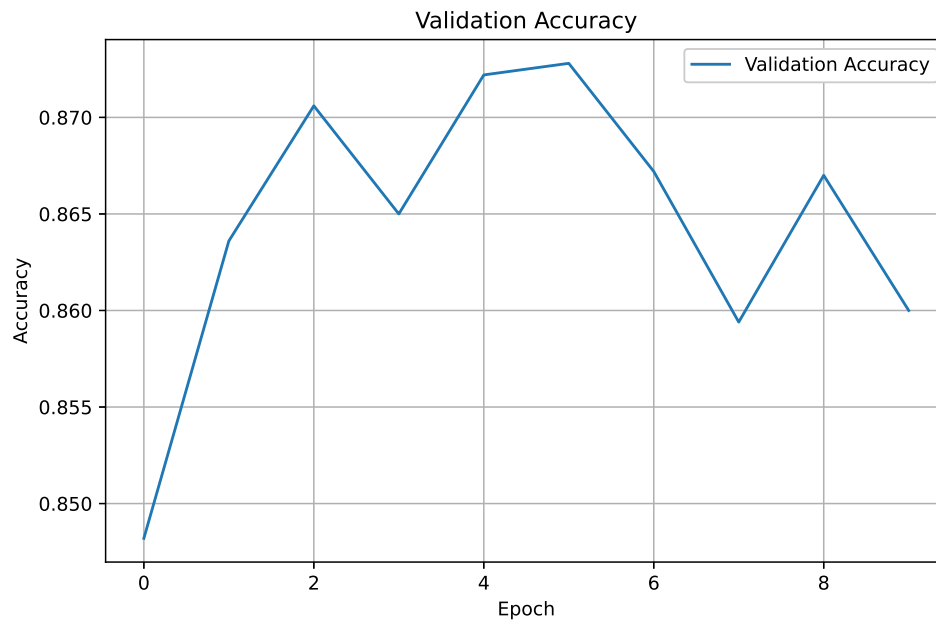


Figure 3: Validation Accuracy

Observation:

The validation accuracy starts improving but attains a value close to 87.3% before varying in the latter epochs. The subsequent fall in the accuracy implies lower generalization.

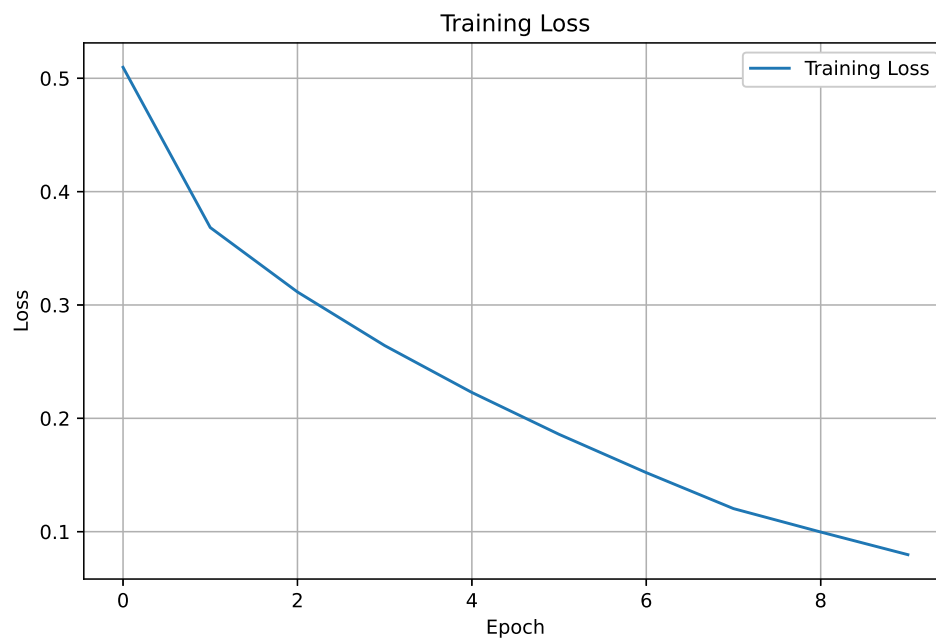


Figure 4: Training Loss

Observation:

The training loss steadily decreases from around 0.51 to 0.08 during the training period. This implies that the model consistently reduces its mistakes while training on the data.

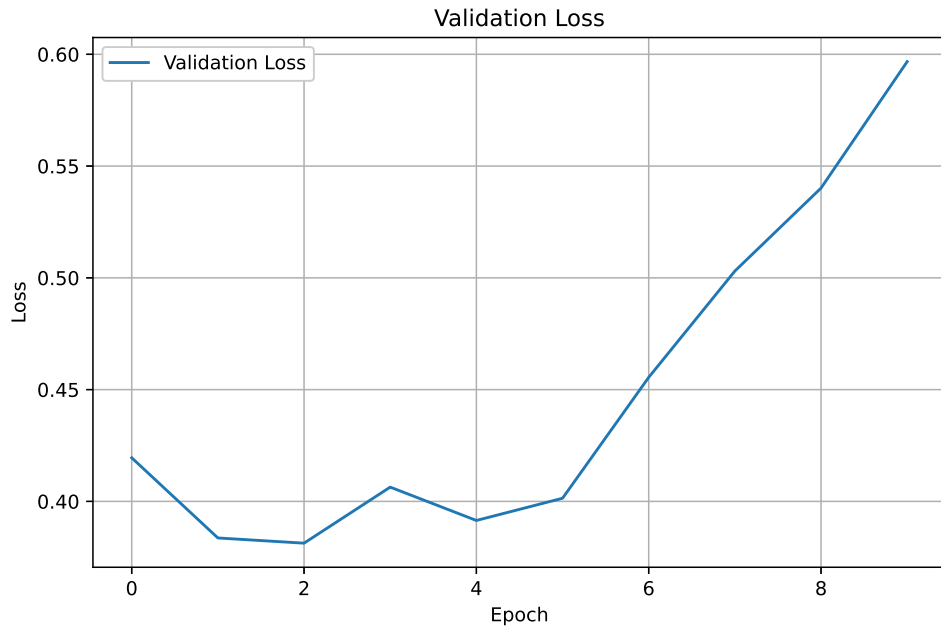


Figure 5: Validation Loss

Observation:

Loss from validation starts off low but soon rises after some epochs and eventually becomes about 0.60. Since the loss is rising as training loss falls, there is overfitting.

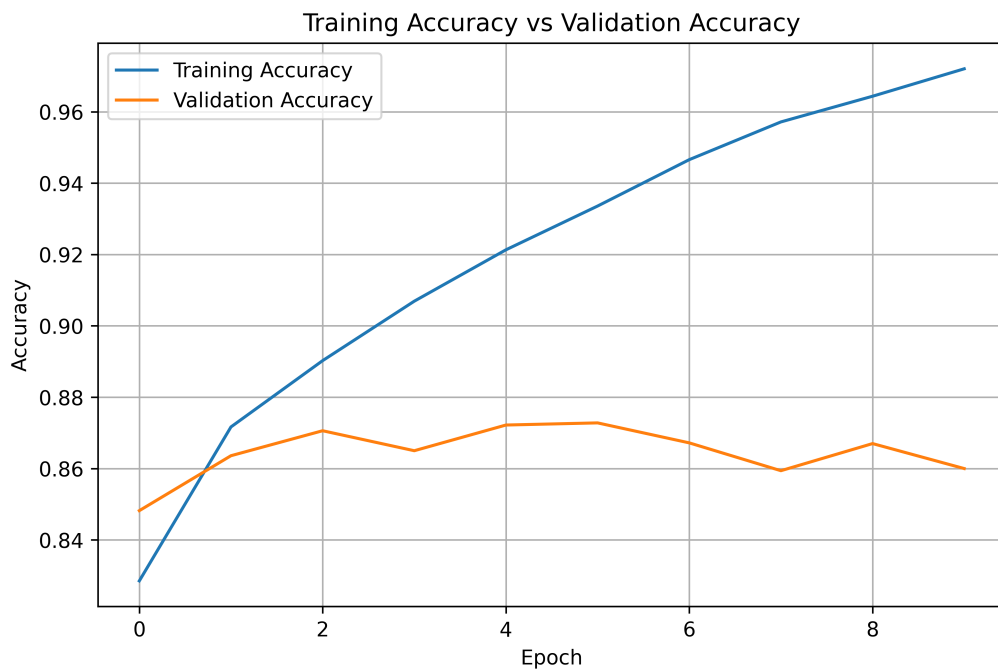


Figure 6: Training Accuracy vs Validation Accuracy

Observation:

Training accuracy increases linearly, whereas validation accuracy attains its maximum value at about the fifth epoch and becomes erratic afterwards. The rising disparity between these two lines signifies overfitting of the model to training data.

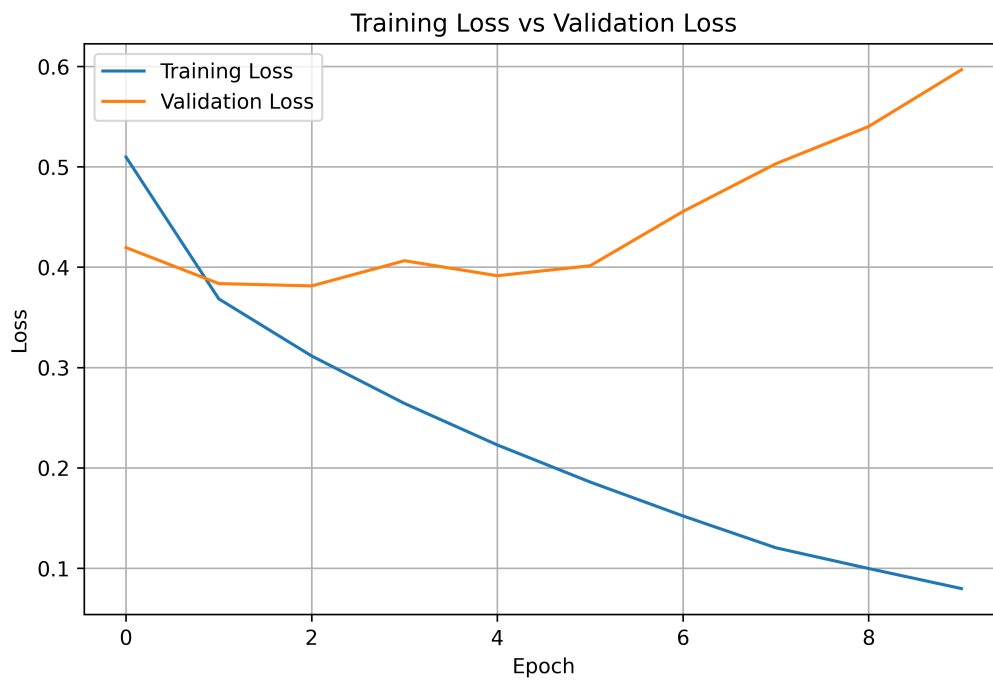


Figure 7: Training Loss vs Validation Loss

Observation:

Training error declines consistently, while validation error shows a decrease followed by an increase substantially. Such differences imply overfitting and poor generalization capability of the model.

Fine Tuning

The final convolutional block of MobileNetV2 was unfrozen for fine tuning while Batch Normalization layers were kept frozen.

Metric	Value
Trainable Parameters	1,044,298
Non-trainable Parameters	1,378,944
Total Parameters	2,423,242

Training Stage	Validation Accuracy (%)
Before Fine Tuning	86.00%
After Fine Tuning	88.82%
Improvement	+2.82%

Observation:

The fine-tuning process resulted in the validation accuracy increasing from 86.00% to 88.82%, hence showing an increase in validation accuracy by 2.82%. This means that fine-tuning increased performance of the pre-trained MobileNetV2 on the CIFAR-10 dataset slightly.

Model Evaluation

Metric	Value
Accuracy	87.99%
Precision	88.48%
Recall	87.99%
F1-score	88.02%

Confusion Matrix



Figure 8: Confusion Matrix

Observation:

From the confusion matrix it can be observed that the test samples have been classified properly, with a good accuracy rate for classes like automobile, frog, ship, and truck. Confusion is higher for those classes that look alike, especially for cats and dogs.

Misclassified Images

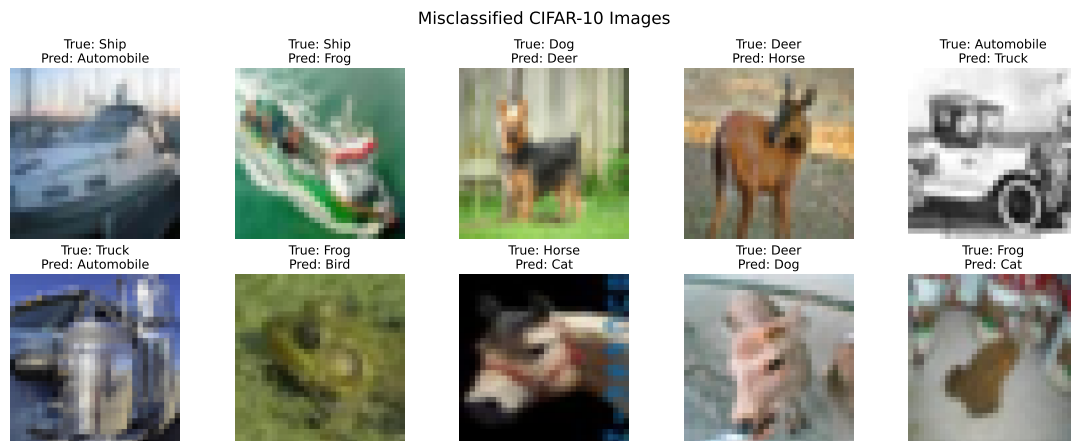


Figure 9: Misclassified CIFAR-10 Images

The 1201 images were misclassified and 10 representative examples are displayed.

Observation:

The classification errors arise from visually similar objects where poor quality of images prevents the demarcation of classes. The classification errors bring out the difficulties that the model faces when differentiating between classes with similar appearances.

Classification Report

Class	Precision	Recall	F1-score	Support
Airplane	0.9290	0.8630	0.8948	1000
Automobile	0.9081	0.9680	0.9371	1000
Bird	0.8263	0.9040	0.8634	1000
Cat	0.7190	0.8520	0.7799	1000
Deer	0.8488	0.8420	0.8454	1000
Dog	0.9159	0.7300	0.8125	1000
Frog	0.9418	0.8740	0.9066	1000
Horse	0.8873	0.9210	0.9038	1000
Ship	0.9376	0.9310	0.9343	1000
Truck	0.9346	0.9140	0.9242	1000
Accuracy	0.8799			10000
Macro Avg	0.8848	0.8799	0.8802	10000
Weighted Avg	0.8848	0.8799	0.8802	10000

Observation:

From the above classification report, we can see that the model has a total accuracy of 87.99% with a macro-average F1-score of 0.8802, which is a sign of a good classification model. The model classifies well on Automobile, Ship, Frog, and Truck, whereas Cat has a relatively low F1-score.

6. Hyperparameter Study

The experiment considered the following hyperparameter values:

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

There are 96 possible combinations. Ten combinations were randomly selected for experimentation.

6.1 Randomized Hyperparameter Search Results

Exp.	LR	Batch	Epochs	Opt.	Dense	Frozen	Train Acc.	Val. Acc.	Test Acc.	Train Loss	Val. Loss	Test Loss	Time (s)
9	0.001	16	20	SGD	128	Partial	99.83	89.78	88.97	0.0285	0.3373	0.3790	498.98
1	0.0001	64	10	Adam	128	Partial	99.36	89.34	88.85	0.0233	0.4031	0.4474	137.09
8	0.001	32	10	Adam	128	Partial	95.49	89.34	88.72	0.1276	0.3792	0.4086	163.51
6	0.001	32	20	SGD	256	Partial	97.71	89.54	88.44	0.0946	0.3180	0.3525	311.39
5	0.001	64	10	Adam	256	Partial	96.60	88.92	88.11	0.0996	0.3953	0.4395	131.50
3	0.001	16	10	Adam	256	Partial	92.54	88.04	87.12	0.2220	0.4110	0.4074	279.10
2	0.001	16	20	SGD	256	All	90.00	86.80	86.23	0.2950	0.3691	0.3978	412.88
7	0.001	32	20	SGD	128	All	87.50	86.48	85.46	0.3634	0.3985	0.4241	292.73
4	0.0001	64	20	SGD	256	All	77.99	78.00	77.55	0.6742	0.6474	0.6763	210.98
10	0.0001	64	10	SGD	256	All	72.84	73.80	73.11	0.8738	0.8580	0.8595	113.89

Best Configurations:

Type	Exp.	LR	Batch	Epochs	Optimizer	Dense	Frozen	Accuracy	Time (s)
Best Accuracy	5	0.001	16	20	SGD	128	Partial	88.97%	498.98
Speed–Accuracy Trade-off	8	0.001	32	10	Adam	128	Partial	88.72%	163.51
Best Balanced	9	0.001	32	20	SGD	256	Partial	88.44%	311.39
Fastest Training	10	0.0001	64	10	SGD	256	All	73.80%	113.89

6.2 Hyperparameter Study Plot

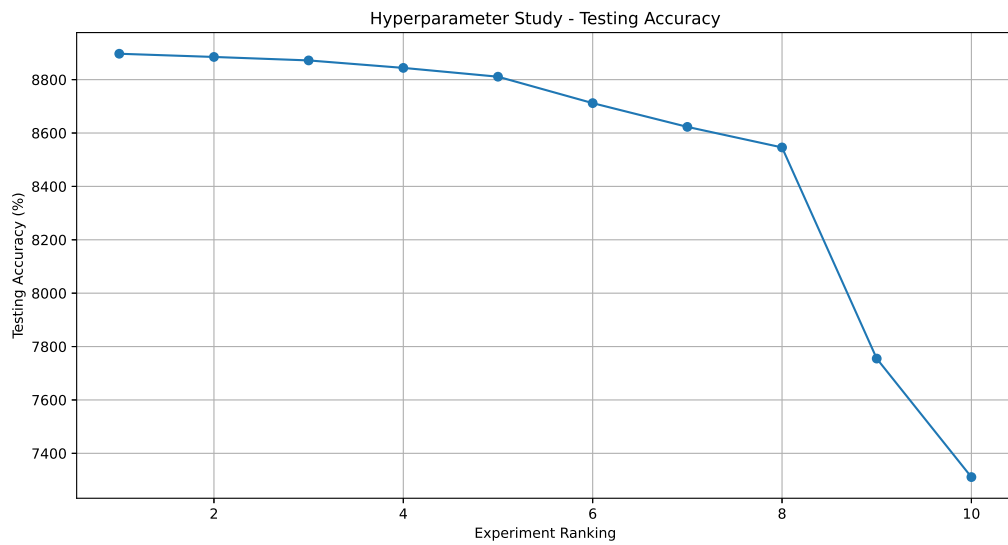


Figure 10: Hyperparameter Study – Testing Accuracy

Observation:

There is a significant difference in the testing accuracies obtained from different hyperparameters setting, which indicates that performance of the model depends on the choice of the hyperparameters. The optimal set of hyperparameters resulted in a testing accuracy of 88.97% with a learning rate of 0.001, batch size 16, SGD optimizer, and 20 epochs and partial freezing.

7. Results

7.1 Performance Metrics

Metric	Value
Training Accuracy	97.20%
Testing Accuracy	87.99%
Precision	88.48%
Recall	87.99%
F1-score	88.02%
Training Time	271.80 seconds
Total Parameters	2,423,242

7.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	83,126	52.06	55.58
AlexNet	8,981,642	10.00	288.93
VGG16	14,781,640	80.82	776.92
GoogleNet	Not Implemented		
ResNet50	23,851,274	85.65	348.93

8. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was considered groundbreaking because it showed that deep convolutional neural networks could perform well in large scale image classification. It used ReLU activation, dropout, and GPU training, improving training efficiency and effectiveness of the model.

2. Why does VGG16 use only 3×3 convolution filters?

VGG16 uses only 3×3 filters to learn small and subtle spatial features without having too many parameters. When multiple 3×3 filters are stacked, an effective receptive field is created, which has more non-linearity and helps learn trends better.

3. Explain the advantages of the Inception module.

In the Inception module, different convolutional processes are conducted simultaneously so that the model can detect features in different sizes. This technique enhances feature detection and saves computational costs using 1×1 convolutions.

4. What is the purpose of residual learning?

With the residual learning model, a deep network can learn the residual which is the difference between the input and the desired output. The use of skip connections enables the deep neural network to overcome the challenges of gradient and information flow through the deep network.

5. Differentiate LeNet and ResNet.

LeNet is a shallow convolutional neural network that was developed primarily for recognizing handwritten digits. ResNet is a deep convolutional neural network designed for performing complex image classifications. ResNet makes use of skip connections to train very deep networks, which is not possible in LeNet.

6. What is Transfer Learning?

Transfer Learning is where the information gained from the learning done by the model in one problem is used in another different but related problem. In this experiment, the transfer of pretrained ImageNet feature extraction to CIFAR-10 classification is demonstrated.

7. Why is fine tuning required?

Fine tuning is necessary to adjust the learned characteristics of the pretrained model to those of the target data set. It could increase the effectiveness of the model by letting selected pretrained layers learn more task-related features.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolutions increase the receptive field through the insertion of gaps between the kernel elements without having to increase the number of kernel parameters. Transposed convolutions increase the spatial dimensions of the feature map and are often used for upsampling.

9. Why do pretrained models converge faster?

Pretrained models converge faster because they have already acquired meaningful features that have been learned using a large set of data. So less training is required for the model to learn features of the new data/problem.

10. Compare the computational complexity of LeNet and ResNet.

LeNet is less computationally complex compared to ResNet as it is shallow and consists of few layers and parameters while ResNet is deep and has millions of parameters.

9. Additional Exercises

9.1 VGG16 Transfer Learning

VGG16 was implemented using ImageNet pretrained weights, with the convolutional base initially frozen.

Metric	Value
Training Accuracy	94.81%
Validation Accuracy	81.84%
Testing Accuracy	80.82%
Training Loss	0.1453
Validation Loss	0.8205
Testing Loss	0.8922
Training Time	776.92 seconds
Total Parameters	14,781,640

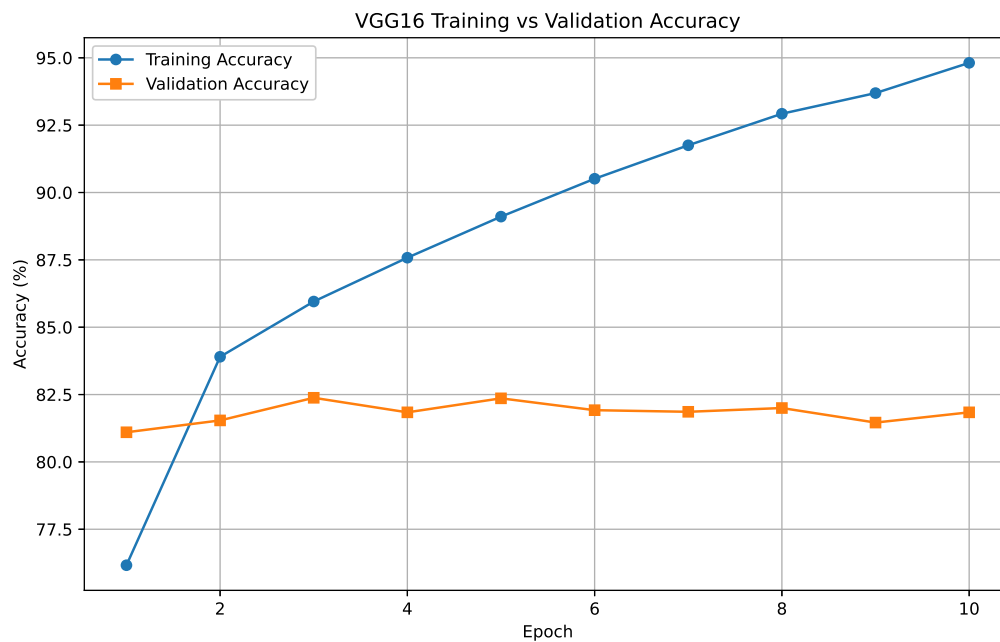


Figure 11: VGG16 Training vs Validation Accuracy

Observation:

Training accuracy has improved significantly, but the validation accuracy is much lower. A big difference between these two curves implies that VGG16 suffers from overfitting on the CIFAR-10 dataset.

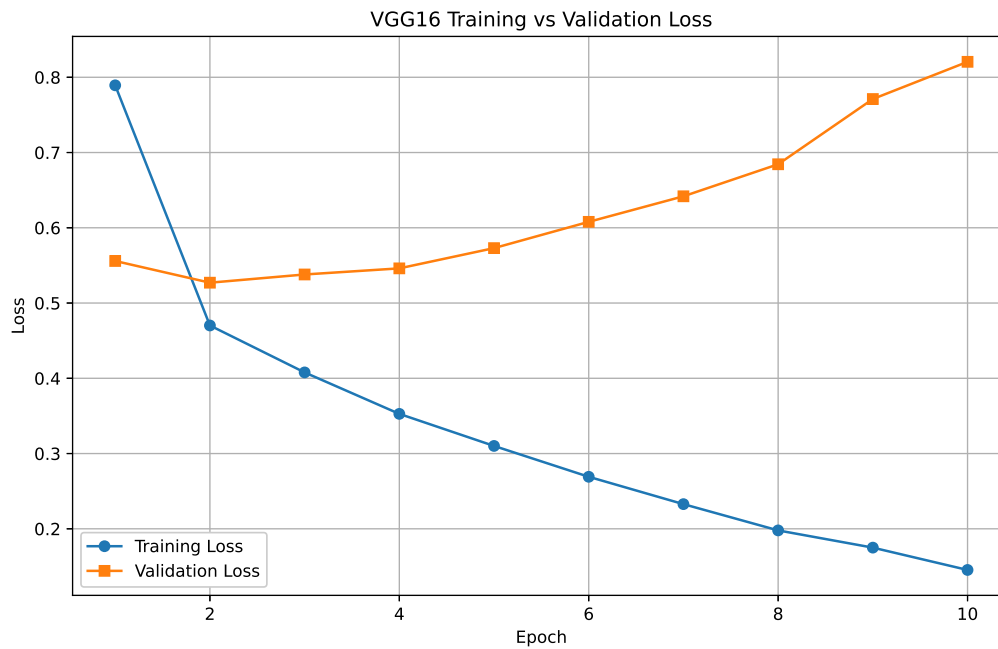


Figure 12: VGG16 Training vs Validation Loss

Observation:

There is a drop in the training loss over time whereas the validation loss stays relatively high. The gap between the two lines clearly shows that the model performs well on training data but poorly on validation data.

9.2 ResNet50 Transfer Learning

ResNet50 was implemented using ImageNet pretrained weights with a Global Average Pooling layer and a 128-unit Dense layer.

Metric	Value
Training Accuracy	96.77%
Validation Accuracy	86.82%
Testing Accuracy	85.65%
Training Loss	0.0890
Validation Loss	0.6297
Testing Loss	0.6633
Training Time	348.93 seconds
Total Parameters	23,851,274

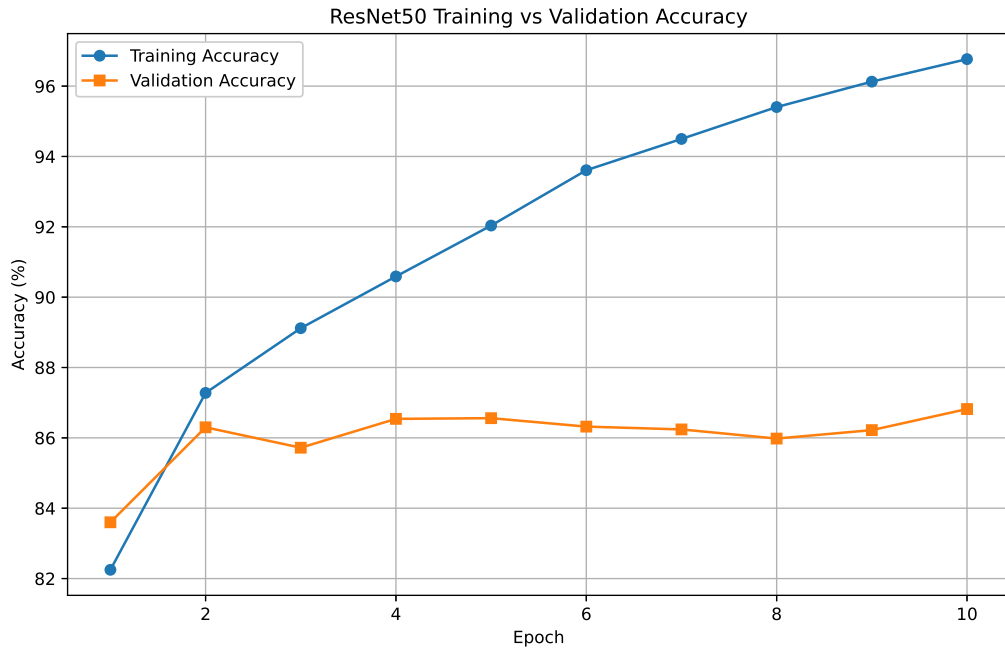


Figure 13: ResNet50 Training vs Validation Accuracy

Observation:

The training accuracy increases incrementally, but the validation accuracy is relatively slow in its improvement. The difference between the two curves suggests overfitting, even though the model does well in classification.

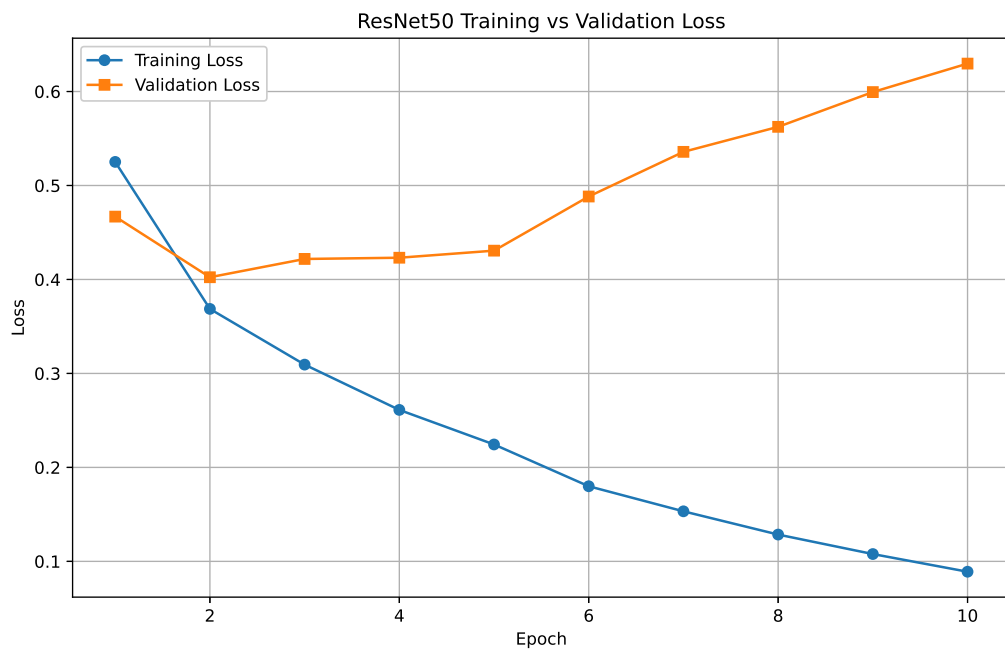


Figure 14: ResNet50 Training vs Validation Loss

Observation:

The training loss reduces steadily, but validation loss is more fluctuating while learning. From this inference,

one can conclude that the model learns better from training data compared to validation data.

9.3 Adam vs SGD

Optimizer	Training Accuracy (%)	Validation Accuracy (%)
Adam	92.53	82.36
SGD	89.20	82.96

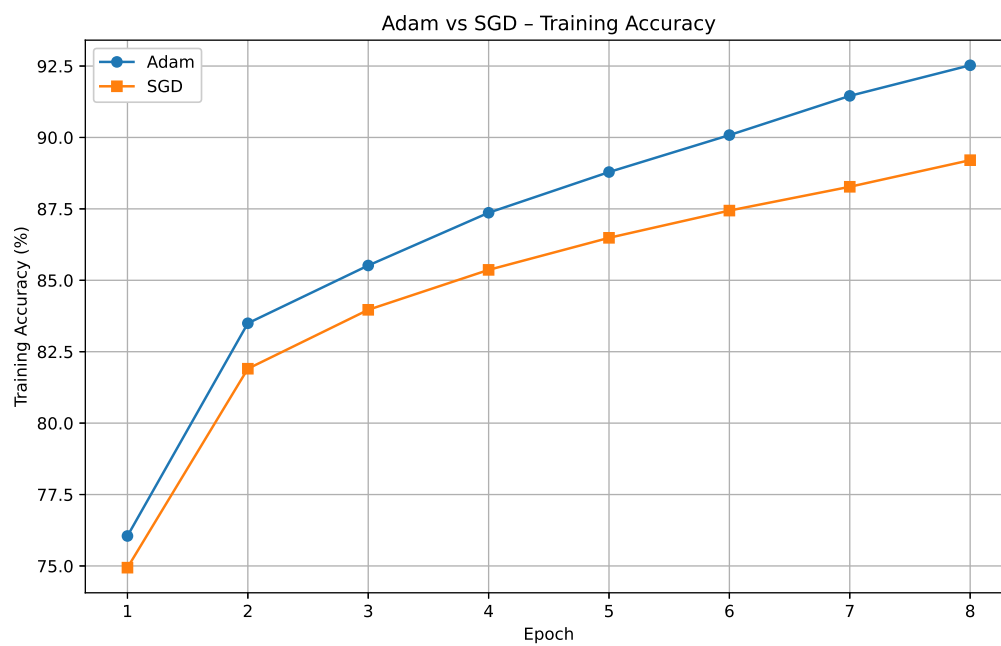


Figure 15: Adam vs SGD – Training Accuracy

Observation:

From the graph showing training accuracy, it is clear that there are different types of convergence with regard to Adam and SGD algorithms.

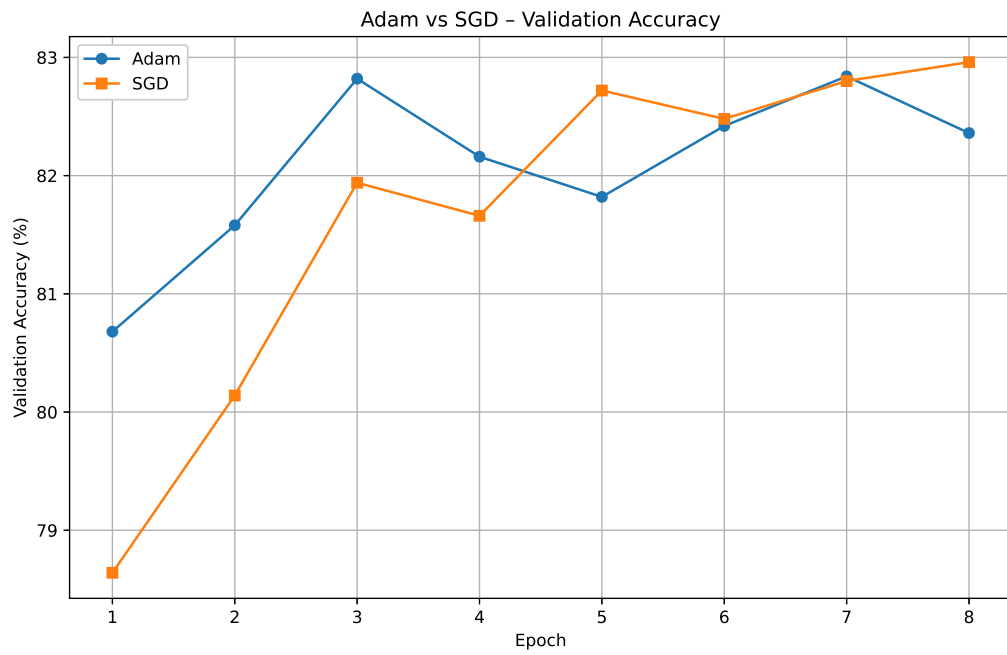


Figure 16: Adam vs SGD – Validation Accuracy

Observation:

The validation accuracy comparison indicates differences in the generalization capabilities of Adam and SGD.

The choice of optimizer influences both speed of convergence and validation accuracy.

9.4 Frozen Layers vs Fine Tuning

Model	Training Strategy	Validation Accuracy (%)
VGG16	Frozen	81.84
VGG16	Fine Tuned	90.88
ResNet50	Frozen	86.82
ResNet50	Fine Tuned	86.88

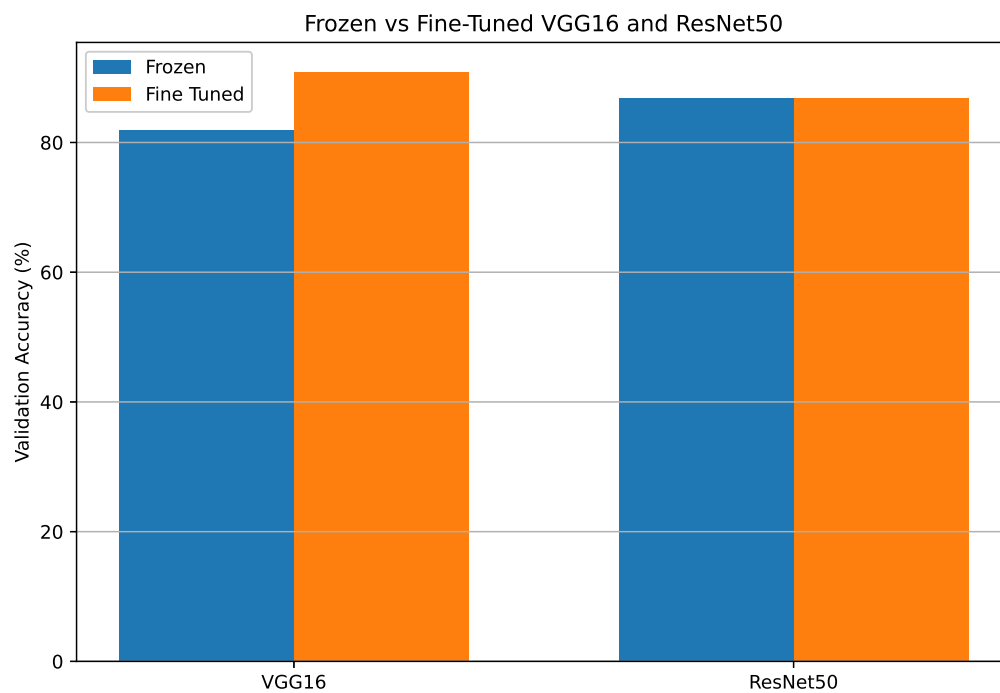


Figure 17: Frozen vs Fine-Tuned VGG16 and ResNet50

Observation:

Fine-tuned models have a greater accuracy than the models that have been frozen. Adapting some layers of pretrained models to the characteristics of the CIFAR-10 dataset makes the model more generalizable to the target

data set.

9.5 Precision, Recall and F1-score

Model	Precision (%)	Recall (%)	F1-score (%)
VGG16	90.83	90.76	90.70
ResNet50	86.24	86.18	86.17

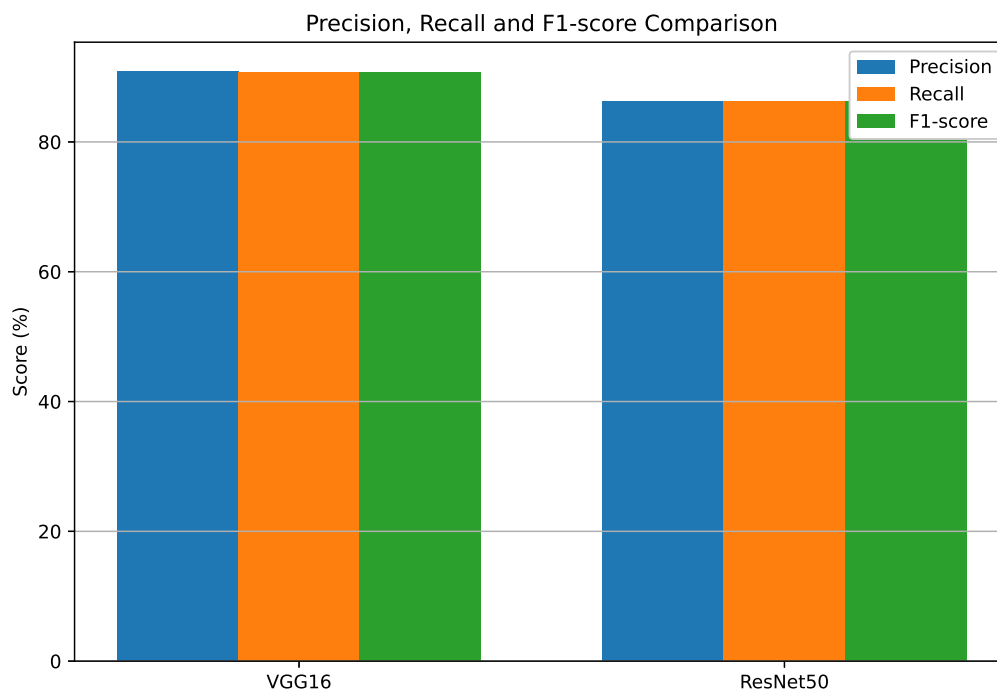


Figure 18: Precision, Recall and F1-score Comparison

Observation:

VGG16 shows better results than ResNet50 in terms of precision, recall, and F1 score. This implies that VGG16 delivers better classification results in this particular setting.

9.6 LeNet-5, AlexNet and ResNet50 Comparison

The implementations in the notebook produced the following comparison:

Architecture	Parameters	Testing Accuracy (%)	Training Time (s)
LeNet-5	83,126	52.06	55.58
AlexNet	8,981,642	10.00	288.93
ResNet50	23,851,274	85.65	348.93

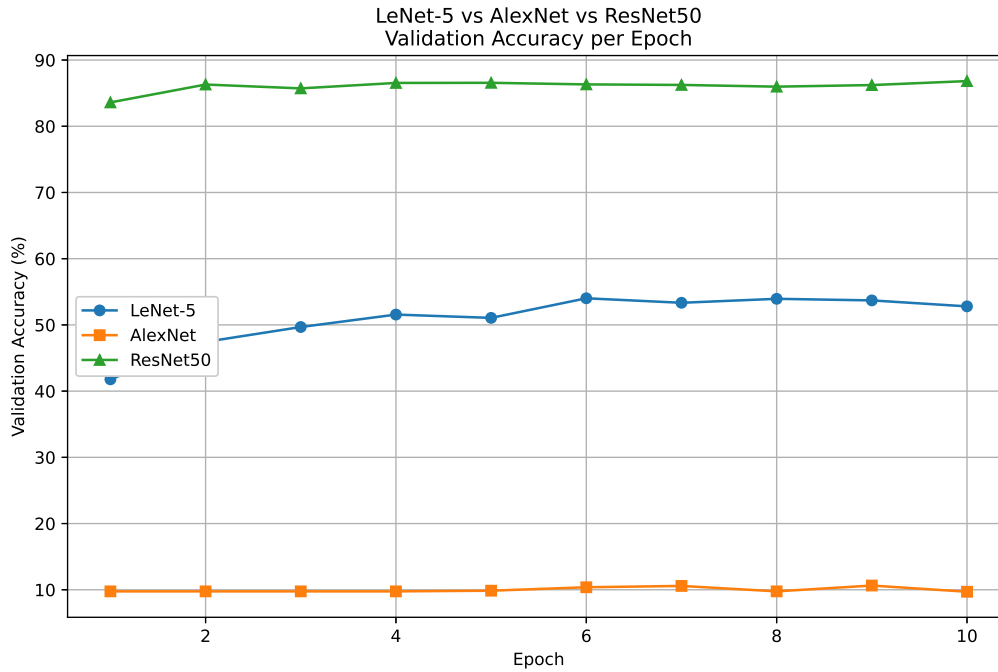


Figure 19: LeNet-5 vs AlexNet vs ResNet50 Validation Accuracy per Epoch

Observation:

ResNet50 has the highest validation accuracy out of the three models built, LeNet-5 comes second. AlexNet has performed poorly in this setup, showing that deeper models do not necessarily perform better.

10. Conclusion

The use of transfer learning through MobileNetV2 was a successful method of classifying images from CIFAR-10 by exploiting the pretrained weights of ImageNet. The process of fine tuning helped to enhance the validation accuracy of the model. Hyperparameters affect the performance of the model, such as the learning rate, batch size, optimization method, number of epochs, and freezing of layers. Evaluation of the model was done using accuracy, precision, recall, F1-score, and confusion matrix. More experiments done through VGG16, ResNet50, other optimization methods, and frozen and fine-tuned models highlighted how the architecture and training method affect classification performance.

11. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>